Project Documentation
Architecture & Requirements

Version 2.0
March 9, 2026

IsoCortex

A High-Performance Localized Neural Information Retrieval Engine

Software Requirement Specification (SRS)

Lead Architect & Developer:

Shaheer Qureshi

TABLE OF CONTENTS

1 Introduction & Project Overview 1
- 1.1 Problem Statement 1
- 1.2 Project Purpose 1
- 1.3 Scope 1
- 1.4 Definitions & Terminology 3

2 Agile Project Management & Product Backlog 3

3 System Architecture 4

4 Functional Requirements 7
- 4.1 Data Extraction & Chunking 8
- 4.2 Vectorization & Handoff 9
- 4.3 Search Execution (C++ Core) 10

5 Non-Functional Requirements 12
- 5.1 Performance & Efficiency 12
- 5.2 Security & Operations 13
- 5.3 Quality & Testing 13

6 Interface & Domain Requirements 15
- 6.1 Interface Requirements 15
- 6.2 Domain Requirements 15

7 Use Cases & User Stories 15
- 7.1 Use Cases 16
- 7.2 Agile User Stories 18

8 Out of Scope 19

9 Assumptions & Constraints 19
- 9.1 Assumptions 19
- 9.2 Constraints 20

10 Diagrams & Supporting Models 21
- 10.1 Context Diagram 21
- 10.2 UML Behavioral Diagrams 21
- 10.3 UML Activity Diagram 21

1 Introduction & Project Overview

1.1 Problem Statement

In the era of large language models and semantic search, businesses and individuals face a critical dilemma when managing unstructured private data. Current market solutions rely heavily on cloud-based vector databases (e.g., Pinecone, Weaviate) and external embedding APIs (e.g., OpenAI). This architecture introduces **severe limitations**:

- **Data Privacy Risks:** Uploading sensitive documents, proprietary source code, private archives, and confidential spreadsheets or presentations to third-party cloud servers violates strict confidentiality and compliance standards (e.g., GDPR, HIPAA, SOC 2).
- **Cost Overhead:** Recurring subscription fees for cloud vector storage and token-based billing for embedding generation create prohibitive costs for local developers and small enterprises.
- **Network Latency & Dependency:** Relying on internet connectivity for semantic retrieval introduces latency and renders the system useless in offline or air-gapped environments.
- **Format Fragmentation:** Enterprise knowledge is distributed across heterogeneous file formats—Word documents, PowerPoint decks, Excel spreadsheets, PDFs, and source code—with no unified local retrieval interface.

1.2 Project Purpose

The purpose of **IsoCortex** is to provide a high-performance, 100% localized Neural Information Retrieval Engine that runs entirely on the user's own machine—no installation as a system service, no background daemon, and no network dependency of any kind. By coupling a flexible Python ingestion pipeline with a custom C++ Hierarchical Navigable Small World (HNSW) search engine, IsoCortex empowers users to perform semantic searches across their private files—spanning the full spectrum of industry file formats—with sub-millisecond latency, zero cloud dependencies, and absolute data sovereignty.

IsoCortex is invoked directly from the command line as a **local developer tool**: the user runs a Python script (or compiled binary) on demand, and all processing terminates when the script exits.

1.3 Scope

IsoCortex is a backend engine and command-line software pipeline focused on the extraction, embedding, and retrieval of unstructured data from a wide array of industry-standard file formats. The system provides:

- **Universal Directory Ingestion:** Parsing of local files across the following supported format categories:

Category	Extensions	Library
Plain Text	`.txt`, `.md`, `.log`	Built-in `open()`
Source Code	`.py`, `.cpp`, `.js`, `.ts`, `.java`, `.c`, `.h`	Built-in `open()`
PDF Documents	`.pdf`	`pymupdf` (fitz)
Word Documents	`.docx`, `.odt`	`python-docx`
Presentations	`.pptx`, `.odp`	`python-pptx`
Spreadsheets	`.xlsx`, `.xls`, `.ods`, `.csv`	`openpyxl` / `pandas`
Email & Web	`.eml`, `.html`, `.htm`	`email` / `BeautifulSoup4`

- **Context-Aware Chunking:** Utilizing natural language processing to divide text into semantic nodes of $\sim$120 words with sentence-boundary awareness.
- **Structured Data Linearization:** Converting tabular data (spreadsheets) and slide-based content (presentations) into coherent natural-language text representations before chunking.
- **Local Vectorization:** Generating 384-dimensional dense vectors using the local `all-MiniLM-L6-v2` transformer model.
- **High-Speed Retrieval:** Executing logarithmic $O(\log N)$ semantic searches via a C++ compiled core engine, bridged to Python via `pybind11`.

1.4 Definitions & Terminology

- **Embedding:** A numerical representation (vector) of text that captures its semantic meaning.
- **HNSW (Hierarchical Navigable Small World):** A state-of-the-art graph-based data structure that allows for rapid approximate nearest neighbour (ANN) search in high-dimensional spaces.
- **Cosine Similarity:** A mathematical metric used to measure how similar two document vectors are, regardless of their magnitude.
- **Context Window:** The maximum number of tokens an AI model can process at one time. For this project the limit is strictly 256 tokens.
- **Linearization:** The process of converting structured or semi-structured data (e.g., spreadsheet rows, presentation slides) into a flat, readable natural-language string suitable for NLP chunking.
- **pybind11:** A lightweight C++11 header-only library that exposes C++ classes and functions as Python extension modules (`.so` on Linux/macOS, `.pyd` on Windows), enabling zero-copy in-process calls between Python and C++.
- **ANN (Approximate Nearest Neighbour):** A class of algorithms that find vectors *approximately* closest to a query vector, trading marginal accuracy for dramatic speed gains.
- **efConstruction / efSearch:** HNSW tuning parameters controlling build-time graph quality and query-time search beam width, respectively.
- **Local Tool:** A program invoked manually by the user from the command line on their own machine. It has no persistent daemon process, no background service, and no installer.

2 Agile Project Management & Product Backlog

To ensure rapid, iterative delivery, the development of IsoCortex is managed using the **Agile Scrum methodology**. All requirements, features, and bugs are tracked dynamically using a centralized Trello Kanban board.

Project Tracking:
The development lifecycle is tracked via a live Scrum board, ensuring transparent progression from architectural planning to C++ implementation and Python integration.

The board is strictly organized into the following **Pipeline** to track the lifecycle of every

feature:

1. **Assets:** Design files, models, architecture diagrams, and SRS documents.
2. **Product Backlog:** The master list of all approved FRs and NFRs (Epics & User Stories).
3. **Sprint Backlog:** Tasks selected for the current active development sprint (e.g., C++ graph logic).
4. **Sprint Dev in Progress:** Tasks actively being coded by the engineering team.
5. **Sprint Dev Completed:** Code is written but pending quality assurance.
6. **Sprint QA in Progress:** Active testing (unit testing, C++ memory leak checks, Python integration checks).
7. **Bugs:** Any defects identified during QA that require developer remediation.
8. **Completed:** Feature successfully merged into the 'main' branch.

3 System Architecture

IsoCortex is a **locally executed, on-demand command-line tool**. It does not run as a background service or system daemon. All processing begins when the user invokes a command and terminates when that command completes. It utilises a **Two-Tier Local Processing Pipeline**, isolating high-level natural language processing tasks from low-level memory management and search execution.

Layer 1: The Python Ingestion Engine (The Sensory Input)

Responsible for recursive directory scanning, format-specific text extraction, structured data linearization, semantic chunking, and model inference. Python is chosen for its superior ecosystem of document parsing and machine learning libraries (e.g., `sentence-transformers`, `python-docx`, `python-pptx`, `openpyxl`, `pymupdf`).

Layer 2: The C++ HNSW Core (The Gray Matter)

Responsible for ingesting the exported binary vector data, constructing the multi-layered HNSW graph in RAM, executing Cosine Similarity mathematical operations, and persisting the index to disk. C++ is utilised to ensure optimal memory allocation and sub-millisecond retrieval speeds. The C++ core is compiled as a native Python extension module via `pybind11`, enabling in-process, zero-file-I/O query execution at runtime without any separate server or daemon.

HNSW Tuning Parameters

The following parameters govern the quality-vs-performance trade-off of the HNSW graph and **must be exposed as configurable values** via `config.json`:

Parameter	Role	Default	Tunable
`M`	Max edges per graph node; controls connectivity	`16`	Yes
`efConstruction`	Build-time beam width; higher = better graph, slower build	`200`	Yes
`efSearch`	Query-time beam width; higher = better recall, slower search	`50`	Yes
`dim`	Vector dimensionality (fixed by model)	`384`	No
`space`	Distance metric (`cosine` or `l2`)	`cosine`	Yes

Technology Stack

Concern	Library / Tool	Rationale
PDF extraction	`pymupdf` (fitz)	Fastest, most robust; handles complex layouts
Word extraction	`python-docx`	Official OOXML support for `.docx`
Presentation extraction	`python-pptx`	Official OOXML support for `.pptx`
Spreadsheet extraction	`openpyxl` / `pandas`	Structure access + row linearization
Web/Email extraction	`BeautifulSoup4` / `email`	HTML tag stripping; RFC 5322 email parsing
Sentence boundary	`spacy` (`en_core_web_sm`)	Accurate sentence tokenization for FR-2
Token counting	`transformers.AutoTokenizer`	Exact sub-word token count for NFR-2
Embedding model	`sentence-transformers`	Local CPU/GPU inference, 384-dim output
C++ bridge	`pybind11`	Zero-copy NumPy array hand-off to C++
Build system	`CMake` $\geq$ 3.15	Compiles C++ extension; no installer needed
Unit testing (Python)	`pytest`	FR validation, chunking correctness
Unit testing (C++)	`GoogleTest`	HNSW correctness, recall benchmarks

Recommended Project Structure

```
isocortex/
├── ingestion/
│   ├── scanner.py ..... FR-1: Recursive directory scan
│   ├── extractor.py ..... FR-1: Format-specific text extraction
│   ├── chunker.py ..... FR-2: Sentence-aware chunking w/ overlap
│   └── embedder.py ..... FR-3: all-MiniLM-L6-v2 inference
├── export/
│   └── serializer.py ..... FR-4: .bin + metadata JSON export
├── core/
│   ├── hnsw.hpp ..... HNSW graph data structure
│   ├── hnsw.cpp ..... Graph construction and search
│   ├── cosine.hpp ..... Cosine similarity (SIMD-optimized)
│   ├── persist.cpp ..... FR-7: Index save/load
│   └── bindings.cpp ..... FR-8: pybind11 Python bindings
├── cli/
│   └── main.py ..... UC-1, UC-2, UC-3: CLI entry point
├── tests/
│   ├── test_chunker.py ..... NFR-5: Chunking unit tests
│   ├── test_extractor.py ..... NFR-5: Format extraction unit tests
│   └── test_search.cpp ..... NFR-6: HNSW recall benchmarks
├── config.json ..... HNSW tuning parameters
├── CMakeLists.txt
├── requirements.txt
└── README.md
```

4 Functional Requirements

4.1 Data Extraction & Chunking

FR-1: Bulk Directory Ingestion

- **Description:** The Python pipeline shall accept a root directory path, recursively traverse all subdirectories, and identify all supported file types as defined in Section 1.3.
- **Inputs:** Local file-system path string provided as a command-line argument.
- **Outputs:** Extracted raw text strings and associated metadata (file name, absolute path, format type, sheet/slide index where applicable).
- **Supported Formats:** `.txt`, `.md`, `.log`, `.py`, `.cpp`, `.js`, `.ts`, `.java`, `.c`, `.h`, `.pdf`, `.docx`, `.odt`, `.pptx`, `.odp`, `.xlsx`, `.xls`, `.ods`, `.csv`, `.eml`, `.html`, `.htm`.
- **Priority:** High

FR-1b: Structured Data Linearization

- **Description:** For structured or semi-structured file formats, the Python pipeline shall convert content into coherent natural-language strings prior to chunking, as follows:
  - **Spreadsheets** (`.xlsx`, `.xls`, `.ods`, `.csv`): Each row shall be linearized as a key-value string using column headers as keys (e.g., *"Name: Alice | Department: Engineering | Salary: 85000"*). Each sheet is processed independently and identified by its sheet name in the metadata.
  - **Presentations** (`.pptx`, `.odp`): Text shall be extracted from all shapes, text boxes, and placeholders per slide. Each slide's content is prefixed with its slide number for traceability (e.g., *"[Slide 3] Q2 Revenue grew by 24% year-on-year..."*).
  - **Word Documents** (`.docx`, `.odt`): Text shall be extracted paragraph-by-paragraph, preserving heading hierarchy as inline labels.
  - **Email** (`.eml`): Subject, sender, date, and decoded body shall be extracted and concatenated into a single document string.
  - **HTML/Web** (`.html`, `.htm`): Boilerplate tags, scripts, and styling shall be stripped; only semantic body text shall be retained.
- **Outputs:** A normalised plain-text string per document (or per sheet/slide) ready for the chunking stage.
- **Priority:** High

FR-2: Sentence-Aware Chunking

- **Description:** The system shall partition extracted text into segments of approximately 120 words. The split must occur at the nearest natural sentence boundary (e.g., period or newline) to preserve semantic meaning.
- **Overlap Constraint:** The system must enforce a 25–30% word overlap (approx. 30 words) between consecutive chunks to maintain contextual continuity.
- **Token Guard:** Before each chunk is passed to the embedding model, its subword token count shall be verified via `transformers.AutoTokenizer`. Any chunk exceeding 256 tokens must be further split automatically to comply with NFR-2.
- **Priority:** High

4.2 Vectorization & Handoff

FR-3: Local Neural Embedding

- **Description:** The Python engine shall process every text chunk through the locally stored `all-MiniLM-L6-v2` transformer model using `sentence-transformers`. Inference must run entirely on the local CPU or GPU; no network calls are permitted at any stage.
- **Outputs:** A 384-dimensional dense floating-point vector for each chunk.
- **Batching:** Chunks shall be processed in configurable batches (default: 64) to maximise CPU/GPU throughput.
- **Priority:** High

FR-4: Interoperability Export

- **Description:** The Python pipeline shall serialise the vector matrix and text metadata to disk after each ingestion run. **CSV is explicitly prohibited** due to floating-point precision loss and excessive file size.
- **Required Format:** A flat binary `.bin` file structured as: `[n_vectors: uint32] [dim: uint32] [vectors: float32 × n × dim]`, accompanied by a co-located `metadata.json` sidecar containing chunk ID, source file path, format type, sheet/slide index, and raw chunk text.
- **Alternate Format:** NumPy `.npy` files are an acceptable alternative for the vector matrix during development, given their lossless precision and trivial C++ interoperability via memory mapping.
- **Outputs:** Two files written to a configurable output directory: `vectors.bin` (or `vectors.npy`) and `metadata.json`.
- **Priority:** Medium

4.3 Search Execution (C++ Core)

FR-5: HNSW Graph Construction

- **Description:** The C++ engine shall read the exported binary vector file and construct an in-memory multi-layered HNSW graph using the configurable parameters defined in Section 3 (`M`, `efConstruction`, `dim`, `space`).
- **Error Handling:** If the binary file is malformed or the declared vector dimensionality does not equal 384, the engine shall emit a structured error message to `stderr` and halt gracefully.
- **Priority:** High

FR-6: Semantic Query Execution

- **Description:** The system shall accept a natural language query string, vectorize it using the locally loaded embedding model, and pass the resulting 384-dimensional vector to the C++ core via `pybind11` bindings (zero file I/O at query time). The C++ core will return the top-K most relevant document chunks based on Cosine Similarity scoring, where K is configurable.
- **Outputs:** A ranked list of relevant text chunks, their Cosine Similarity scores, and their source file paths (including sheet/slide index for structured formats), printed to `stdout`.
- **Priority:** High

FR-7: Index Persistence

- **Description:** The C++ engine shall serialise the fully constructed HNSW graph to a binary index file on disk (e.g., `isocortex.index`). On subsequent invocations of the search command, the engine shall reload the saved index from disk without requiring re-ingestion of source documents.
- **Outputs:** A persistent binary index file written to a configurable path on the local file system.
- **Incremental Insertion:** If new documents are ingested, the system shall support inserting new nodes into the existing in-memory index without requiring a full rebuild, where algorithmically feasible, before re-saving to disk.
- **Priority:** High

FR-8: Python–C++ Runtime Bridge

- **Description:** The C++ HNSW engine shall be compiled as a native Python extension module using `pybind11`. The compiled `.so` (Linux/macOS) or `.pyd` (Windows) file is placed in the `core/` directory and imported directly by the Python scripts—no system-wide installation is required. The module shall expose at minimum the following interface:
  - `build_index(vectors: np.ndarray, config: dict) → None`
  - `save_index(path: str) → None`
  - `load_index(path: str) → None`
  - `search(query_vector: np.ndarray, k: int) → List[Tuple[int, float]]`
- **Zero-Copy Constraint:** Vector data shall be transferred as `float32` NumPy arrays using `pybind11`'s buffer protocol to eliminate serialisation overhead.
- **Build:** The extension is compiled once by the developer via `cmake --build build/` from the project root. The resulting module file is used in-place; no package manager or system-wide installation step is needed.
- **Priority:** High

5 Non-Functional Requirements

5.1 Performance & Efficiency

NFR-1: Retrieval Latency

- **Description:** Once the HNSW graph is loaded into RAM, the C++ search execution (FR-6) must achieve sub-millisecond retrieval speeds for single-query execution on an index of up to 100,000 vectors.
- **Measurability:** Benchmarked via internal C++ `std::chrono` timer modules; query execution time is appended to `benchmark.log` in the project directory.

NFR-2: Context Integrity

- **Description:** Zero text truncation is permitted. The chunking algorithm (FR-2) must guarantee that no generated segment exceeds the 256-token limit of the `all-MiniLM-L6-v2` model. This applies equally to chunks derived from all supported file formats, including linearized spreadsheet rows and slide text.
- **Measurability:** Enforced by the token guard in FR-2; violations are logged as `[ERROR]` to `stderr` and the offending chunk is re-split automatically before being passed to the embedding model.

5.2 Security & Operations

NFR-3: Absolute Privacy (Air-Gapped Operation)

- **Description:** 100% of data processing—including format extraction, linearization, model inference, and graph searching—must occur locally on the host machine. The system must not make any outbound network requests at any point during ingestion, indexing, or querying. This is enforced by design: the tool has no network client code and all model weights are loaded from the local file system.
- **Verification:** All third-party Python libraries must be pinned to specific versions in `requirements.txt` and installed into a local virtual environment (`venv`) from a local package cache where required for air-gapped deployment.

NFR-4: Zero Operational Cost

- **Description:** The architecture must completely bypass commercial APIs, ensuring a $0 recurring operational cost for the end-user. All libraries used must be open-source with permissive licences (MIT, Apache 2.0, or BSD).

5.3 Quality & Testing

NFR-5: Unit & Integration Testing

- **Description:** The system shall include automated tests covering:
  - **Chunking correctness (Python / pytest):** Verify overlap percentage, sentence-boundary respect, and token-count compliance for all supported format outputs.
  - **Format extraction (Python / pytest):** Verify correct text extraction from sample `.docx`, `.pptx`, `.xlsx`, `.csv`, `.pdf`, and `.eml` fixture files stored in `tests/fixtures/`.
  - **C++ HNSW correctness (GoogleTest):** Verify graph construction, serialisation/deserialisation round-trips, and search result validity on a known toy dataset.
- **Execution:** Tests are run locally via `pytest tests/` (Python) and `ctest` (C++) from the project root. No CI server is required for MVP.
- **Priority:** Medium

NFR-6: HNSW Recall Benchmarks

- **Description:** The C++ HNSW search shall achieve a minimum **Recall@10 of 95%** as measured against brute-force exact nearest-neighbour results on a representative 10,000-vector benchmark dataset.
- **Measurability:** The benchmark script `tests/test_search.cpp` generates ground truth via an exhaustive cosine scan and compares it against HNSW results, printing a recall report to `stdout`.
- **Priority:** Medium

NFR-7: Index Build Latency

- **Description:** HNSW graph construction time shall be measured and reported to the terminal upon completion of the index command. Target: build an index of 10,000 vectors in under 30 seconds on a commodity CPU (e.g., Intel Core i5, 8 GB RAM).
- **Measurability:** Build duration measured via `std::chrono` and reported as: `[IsoCortex] Index built: N vectors in X.XXs`.
- **Priority:** Low

6 Interface & Domain Requirements

6.1 Interface Requirements

- **UI-1: Command Line Interface (CLI):** The primary interface for IsoCortex shall be a terminal-based CLI executed via Python (‘python cli/main.py‘). It must support distinct operational commands (e.g., ‘index‘, ‘search‘) and robust argument parsing.
- **API-1: C++ Runtime Bridge:** The C++ engine must be strictly wrapped using ‘pybind11‘ to allow Python to call graph construction and search functions directly without executing external binary subprocesses.
- **HW-1: Local Hardware Execution:** The system must run entirely on the user’s local CPU (or local CUDA-enabled GPU if configured). It requires local read/write permissions for the target directories and the internal ‘isocortex.index‘ file.

6.2 Domain Requirements

- **DR-1: Operating System Agnosticism:** The Python pipeline and C++ CMake build system must be cross-platform, capable of running on Windows, macOS, and Linux environments without requiring OS-specific source code modifications.
- **DR-2: File Permission Respect:** The ingestion scanner must gracefully handle read-only files and directory permission errors without crashing the global indexing process.

7 Use Cases & User Stories

7.1 Use Cases

UC-1: Indexing a Local Directory

Actor: User (developer / analyst operating the local machine)

Trigger: User executes the index script from a terminal.

Main Flow:

1. User opens a terminal and runs: `python cli/main.py index ./my_documents`
2. The Python ingestion engine recursively scans the directory, identifying all supported file types.
3. Format-specific extractors parse each file; structured formats (spreadsheets, presentations) are linearized into natural-language strings.
4. The text is chunked with sentence-boundary enforcement and enforced overlap; each chunk is token-validated before embedding.
5. All chunks are vectorized in batches via the locally loaded `all-MiniLM-L6-v2` model.
6. The vector matrix and metadata sidecar are written to `vectors.bin` and `metadata.json` in the configured output directory.
7. The C++ engine (loaded via `pybind11`) reads the binary file, constructs the HNSW graph in RAM, and saves the persistent index to `isocortex.index`.
8. The terminal reports:
   `[IsoCortex] Indexed N chunks from M files in X.Xs.`

Alternate Flow: If an unsupported or corrupted file is encountered, the Python engine skips it, logs a `[WARNING]` message to `stderr` with the file path and reason, and continues without crashing.

UC-2: Executing a Semantic Query

Actor: User

Trigger: User executes the search script from a terminal.

Main Flow:

1. User runs:
   `python cli/main.py search "Where is the Q3 revenue data?"`
2. Python loads the locally cached model and vectorizes the query string into a 384-dimensional vector.
3. The query vector is passed in-process to the C++ HNSW core via `pybind11` bindings (zero disk I/O at query time).
4. The C++ HNSW graph navigates its multi-layered nodes to find the top-K closest vectors by Cosine Similarity.
5. The system resolves chunk IDs to text and source metadata via `metadata.json`.
6. Results are printed to `stdout` ranked by relevance, displaying chunk text, source file path, format type, sheet/slide reference, and similarity score.

Alternate Flow: If `isocortex.index` is not found, the system prints:
`[ERROR] No index found. Run 'python cli/main.py index <path>' first.`

UC-3: Incremental Re-Indexing

Actor: User

Trigger: User adds new files and runs the index script with the `-incremental` flag.

Main Flow:

1. User adds new files to the previously indexed directory.
2. User runs:
   `python cli/main.py index ./my_documents -incremental`
3. The scanner compares file paths and modification timestamps against those stored in `metadata.json`, identifying only new or modified files.
4. New chunks are extracted, linearized (if applicable), embedded, and inserted into the existing in-memory HNSW graph.
5. The updated index is written to disk, overwriting the prior `isocortex.index`.
6. The terminal reports the number of new chunks added.

Note: Full re-indexing from scratch is always available via the `-full` flag: `python cli/main.py index ./path -full`

7.2 Agile User Stories

US-1: Bulk Directory Scan

"As a developer, I want to point the tool at an entire repository folder, so that it automatically ingests all my Python and C++ source code without me having to select files individually."

US-2: Offline Semantic Search

"As a data-sensitive analyst, I want to run natural language queries against my local Excel and PDF files without an internet connection, so that my proprietary company data remains strictly confidential."

US-3: C++ Engine Speed

"As an end-user, I want the search results to be returned almost instantaneously, so that I can quickly iterate my queries without waiting for slow Python loops."

US-4: Python-C++ Bridge

"As the software architect, I want the C++ code to be compiled directly as a Python module via pybind11, so that I can call 'engine.search()' inside my Python script without dealing with clunky command-line subprocesses."

8 Out of Scope

System Exclusions

- **OOS-1: Cloud Integrations:** IsoCortex will not integrate with OpenAI, Anthropic, or external vector databases such as Pinecone or Weaviate.
- **OOS-2: Real-Time Folder Monitoring:** The system will not run background daemon processes to watch for live file edits. Re-indexing must be manually triggered by the user. UC-3 covers incremental re-indexing on demand.
- **OOS-3: Graphical User Interface:** The MVP relies exclusively on a command-line interface. No web server, Electron app, or desktop GUI is included.
- **OOS-4: Legacy Binary Office Formats:** The legacy binary formats `.doc` (Word 97–2003) and `.ppt` (PowerPoint 97–2003) are not supported in MVP v1.0. Users must convert such files to `.docx`/`.pptx` before ingestion.
- **OOS-5: Image & Audio Extraction:** OCR on scanned PDFs or image files (`.png`, `.jpg`) and speech-to-text on audio/video files are out of scope for v1.0.
- **OOS-6: Distributed / Multi-Node Search:** Sharding the HNSW index across multiple machines is out of scope for the initial release.
- **OOS-7: Installer or Package Distribution:** IsoCortex is not distributed as an installable package, system service, or executable installer. It is cloned from source and run directly.

9 Assumptions & Constraints

9.1 Assumptions

- **ASM-1: Hardware:** The target machine has sufficient RAM to hold the HNSW graph in memory during search operations. A minimum of 4 GB free RAM is assumed for indices up to 100,000 vectors.
- **ASM-2: Dependencies:** The host machine has Python 3.9+, a modern C++ compiler (GCC $\geq$ 9 or Clang $\geq$ 10) with C++17 support, and CMake $\geq$ 3.15 installed. Python dependencies are installed into a local virtual environment via `pip install -r requirements.txt`.
- **ASM-3: File Encoding:** Plain-text files (`.txt`, `.md`, `.log`, source code) are assumed to be UTF-8 encoded. Files using legacy encodings (e.g., Latin-1, Windows-1252) may require pre-processing conversion.
- **ASM-4: Office Format Versions:** Spreadsheet and presentation files are assumed to be in OOXML-compliant formats (`.xlsx`, `.pptx`), the default output of Microsoft Office 2007+ and all versions of LibreOffice.

- **ASM-5: Offline Model:** The `all-MiniLM-L6-v2` model weights are pre-downloaded to the local `sentence-transformers` cache directory before any air-gapped use. No internet connection is required at runtime.

9.2 Constraints

- **CON-1: Context Limits:** All chunks are strictly bound by the 256-token limit of the `all-MiniLM-L6-v2` transformer, regardless of source format.
- **CON-2: Language Support:** The initial sentence-boundary detection and chunking logic is optimised for English text and standard programming languages. The AI model handles multilingual text reasonably well, but tokenization quality for non-Latin scripts may vary.
- **CON-3: Spreadsheet Size:** Extremely wide spreadsheets (e.g., 500+ columns) may produce linearized rows exceeding the 256-token limit; the token guard in FR-2 will split them automatically. Column selection or row sampling may be advisable for very large datasets.
- **CON-4: Build Step Required:** The `pybind11` C++ extension (FR-8) must be compiled by the developer before first use via `cmake --build build/` from the project root. The resulting module is used in-place within the repository; no system-wide installation is performed.
- **CON-5: Single-Machine Operation:** IsoCortex is designed to run on a single local machine. All paths, index files, and model weights are resolved relative to the local file system.

10 Diagrams & Supporting Models

This section provides placeholders for visual representations of the system's high-level boundaries and behavioral interactions.

10.1 Context Diagram

The Context Diagram illustrates the high-level interactions between the user, the core CLI system, and the local file system.

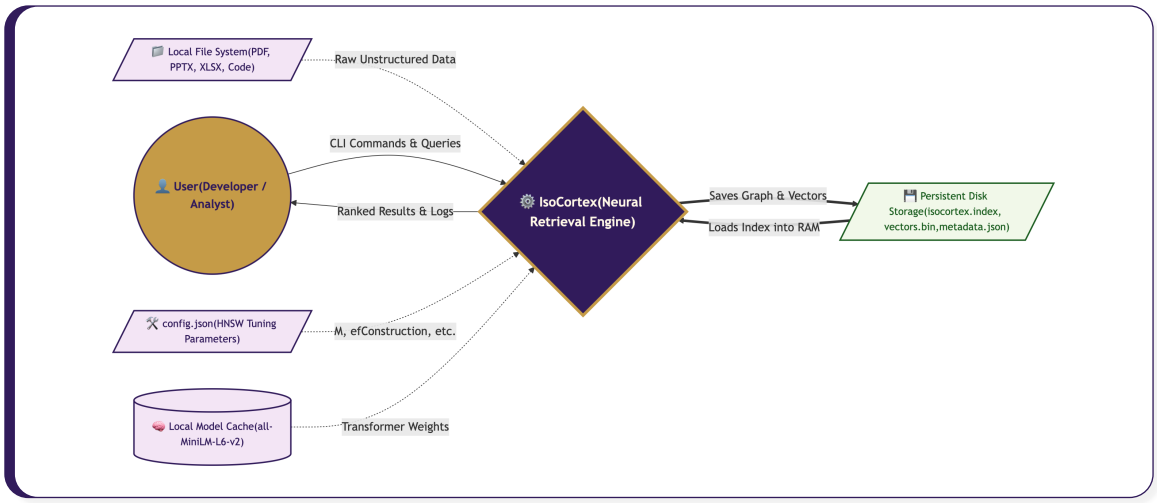

Figure 1: *High-Level Context Diagram of the IsoCortex System*

10.2 UML Behavioral Diagrams

The Use Case diagram captures the functional requirements from the perspective of the developer/analyst operating the CLI.

System Use Cases

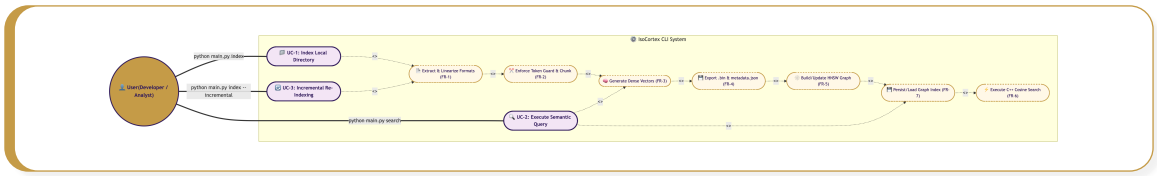

Figure 2: *Use Case Diagram: Local User Interactions*

10.3 UML Activity Diagram

The Activity Diagram illustrates the step-by-step logic of the two-tier ingestion pipeline (Python Extraction → C++ HNSW Graph Build).

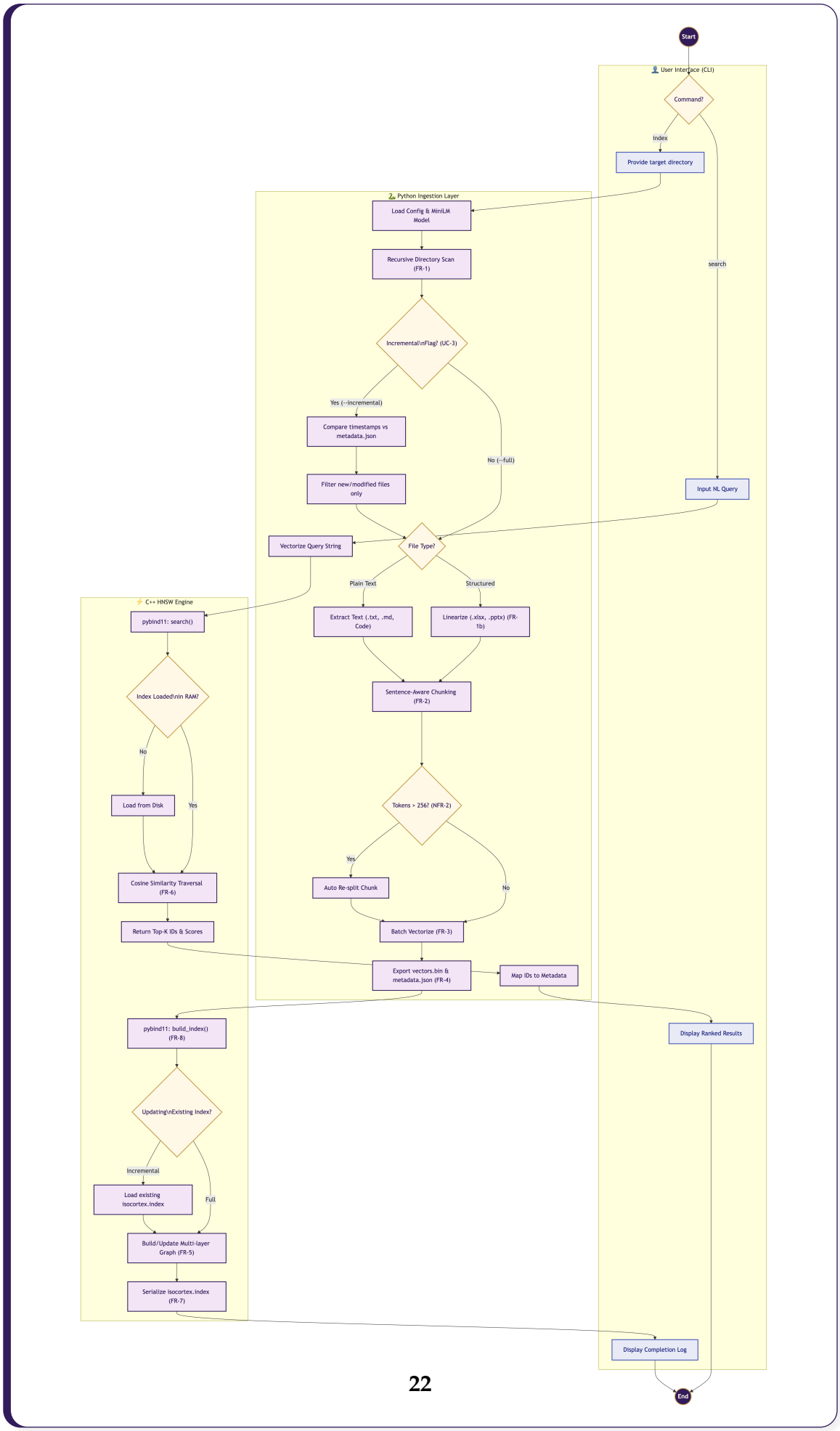

Figure 3: *Activity Diagram: Ingestion and Indexing Workflow*